

```
In [178... #Importing all the required libraries and data for Time Series Analysis
from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve

        local, _ = urlretrieve(url, filename)
        print("Downloaded " + local)

download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkst
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkpl
```

```
In [180... # Import additional functions required to get Correlation and AutoCorrelatio

import thinkstats2
import thinkplot
import numpy as np
import pandas as pd
```

```
In [182... try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
```

```
In [184... download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/nsfg.py
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2002Fem
```

```
In [186... import nsfg, warnings

warnings.simplefilter(action='ignore', category=FutureWarning)

preg = nsfg.ReadFemPreg()
complete = preg.query("outcome in [1, 3, 4]").prglngth
cdf = thinkstats2.Cdf(complete, label="cdf")
```

```
In [187... download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/surviva
```

```
In [188... import survival

def MakeSurvivalFromCdf(cdf, label=""):
    """Makes a survival function based on a CDF.
    cdf: Cdf
    returns: SurvivalFunction
```

```

"""
ts = cdf.xs
ss = 1 - cdf.ps
return survival.SurvivalFunction(ts, ss, label)

```

```

In [189... download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/1995Fem
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2006_20
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/2006_20

```

```

In [190... # reading data for survival for different decades
resp5 = survival.ReadFemResp1995()
resp6 = survival.ReadFemResp2002()
resp7 = survival.ReadFemResp2010()

```

```

In [191... # making an array out of the above data
resps = [resp5, resp6, resp7]

```

Exercise: In NSFG Cycles 6 and 7, the variable `cmdivorcx` contains the date of divorce for the respondent's first marriage, if applicable, encoded in century-months.

Compute the duration of marriages that have ended in divorce, and the duration, so far, of marriages that are ongoing. Estimate the hazard and survival curve for the duration of marriage.

Use resampling to take into account sampling weights, and plot data from several resamples to visualize sampling error.

Consider dividing the respondents into groups by decade of birth, and possibly by age at first marriage.

```

In [213... def CleanData(resp):
    """Cleans respondent data.

    resp: DataFrame
    """
    resp["cmdivorcx"] = resp.cmdivorcx.replace([9998, 9999], np.nan)
    resp["notdivorced"] = resp.cmdivorcx.isnull().astype(int)
    resp["duration"] = (resp.cmdivorcx - resp.cmmarrhx) / 12.0
    resp["durationsofar"] = (resp.cmintvw - resp.cmmarrhx) / 12.0

    month0 = pd.to_datetime("1900-01-01")
    dates = [month0 + pd.DateOffset(months=cm) for cm in resp.cmbirth]
    resp["decade"] = (pd.DatetimeIndex(dates).year - 1900) // 10

```

```

In [215... CleanData(resp6)
married6 = resp6[resp6.evrmarry == 1]

```

```
CleanData(resp7)
married7 = resp7[resp7.evrmarry == 1]
```

In [228... *# Solution*

```
def ResampleDivorceCurve(resps):
    """Plots divorce curves based on resampled data.

    resps: list of respondent DataFrames
    """
    for _ in range(11):
        samples = [thinkstats2.ResampleRowsWeighted(resp) for resp in resps]
        sample = pd.concat(samples, ignore_index=True)
        PlotDivorceCurveByDecade(sample, color="#225EA8", alpha=0.1)

    thinkplot.Show(xlabel="years", axis=[0, 28, 0, 1])
```

In [230... *# Solution*

```
def ResampleDivorceCurveByDecade(resps):
    """Plots divorce curves for each birth cohort.

    resps: list of respondent DataFrames
    """
    for i in range(41):
        samples = [thinkstats2.ResampleRowsWeighted(resp) for resp in resps]
        sample = pd.concat(samples, ignore_index=True)
        groups = sample.groupby("decade")
        if i == 0:
            survival.AddLabelsByDecade(groups, alpha=0.7)

        EstimateSurvivalByDecade(groups, alpha=0.1)

    thinkplot.Config(xlabel="Years", ylabel="Fraction undivorced", axis=[0,
```

In [232... *# Solution*

```
def EstimateSurvivalByDecade(groups, **options):
    """Groups respondents by decade and plots survival curves.

    groups: GroupBy object
    """
    thinkplot.PrePlot(len(groups))
    for name, group in groups:
        _, sf = EstimateSurvival(group)
        thinkplot.Plot(sf, **options)
```

In [234... *# This function used to define survival estimations*

```
def EstimateSurvival(resp):
    """Estimates the survival curve.

    resp: DataFrame of respondents

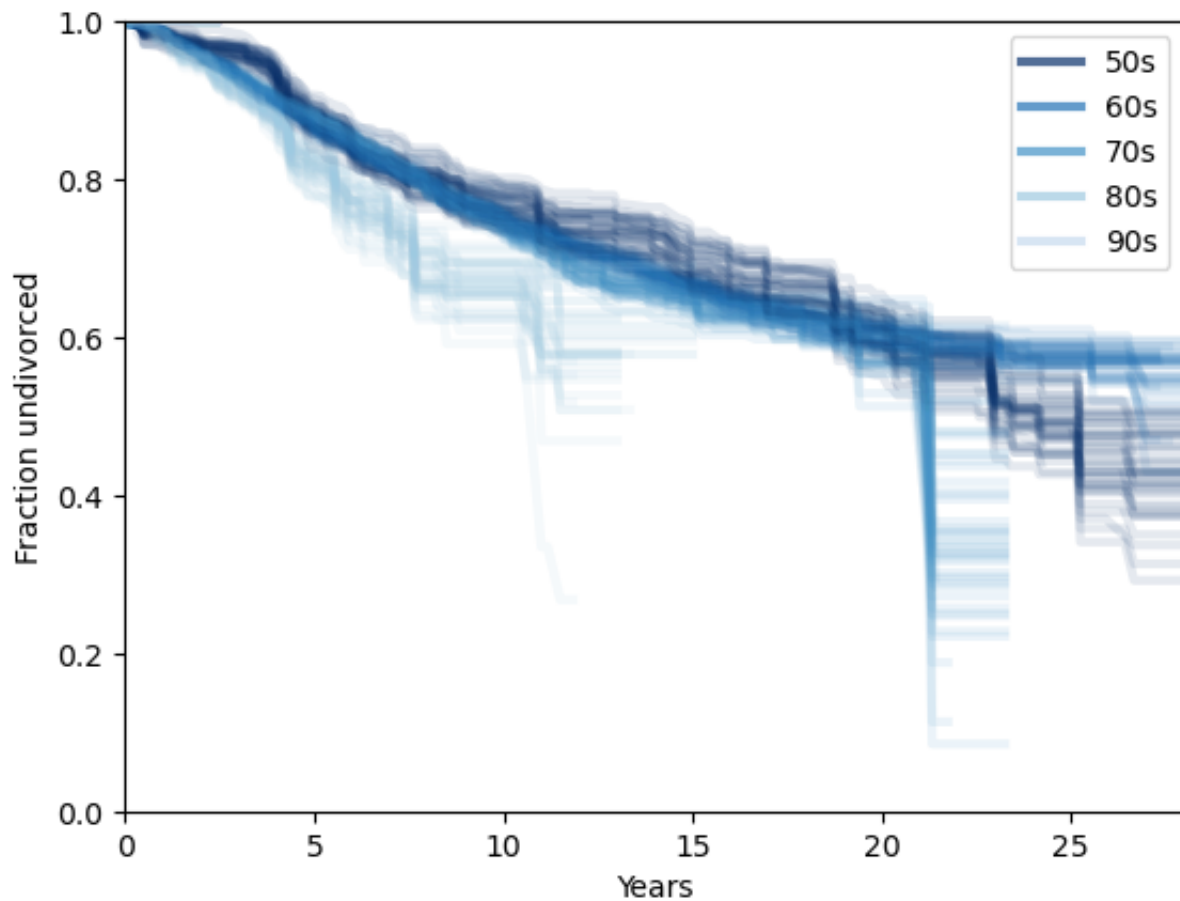
    returns: pair of HazardFunction, SurvivalFunction
    """
    complete = resp[resp.notdivorced == 0].duration.dropna()
    ongoing = resp[resp.notdivorced == 1].durationsofar.dropna()

    hf = survival.EstimateHazardFunction(complete, ongoing)
    sf = hf.MakeSurvival()

    return hf, sf
```

In [236... *# Solution*

```
ResampleDivorceCurveByDecade([married6, married7])
```



In []:

In []: